

Implementación IA para Pacman

Pedro Morales Méndez

Septiembre 2026

1 Introducción

Pacman es uno de los juegos de arcade más conocidos y amados de la historia, teniendo un ciclo de juego bastante simple y fácil de entender, pero no eso no significa que el comportamiento de los enemigos sea fácil, ya que estos dependen de tomar decisiones en base a ciertos factores, además de que estos también dependen del mapa, complejizando más su comportamiento en vez de simplemente tomar decisiones en el momento usando sólo números.

Para este proyecto se intentó imitar ese comportamiento característico de los fantasmas usando 2 modelos matemáticos distintos en funcionamiento base, pero, hasta cierto grado, ser bastante eficientes. Los resultados finales no son una forma para "demostrar" cual de los 2 modelos es mejor que otro, ya que ambos son buenos, pero en sus distintos aspectos.

El uso de IA externa, como Claude, fue utilizada exclusivamente para resolver problemas de código, pero no fue involucrada en la programación principal de las Máquinas de Estado Finitas y Árbol de Comportamiento, ni tampoco en el diseño de transiciones, condiciones y estados.

2 Implementación

2.1 Máquina de Estado Finita (FSM)

Consiste en tener un número finito de estados donde cada uno contiene una o más transiciones a otros estados dentro de la misma máquina. Cada transición, a su vez, consiste de una revisión constante de una condición, pero esta revisión sólo aplica las transiciones del estado actual y no de todas las transiciones de la máquina. En el momento en el que una condición de transición se cumple, la máquina cambiara el estado actual al nuevo estado que devuelve la transición, repitiendo la lógica nuevamente.

Es fácil confundirse con los estados y sus transiciones, más aún cuando una transición puede estar presente en 2 o más estados, haciendo que se tengan que crear transiciones específicas para cierto estados. Por eso mismo siempre es mejor crear un diagrama para visualizar los estados y sus conexiones.

2.2 Árbol de Comportamiento (BT)

Su funcionamiento depende de "nodos", donde cada nodo hace algo dentro del árbol, por ejemplo, un nodo Secuencia, que revisa cada hijo y sólo tira SUCCESS cuando todos sus hijos lo hacen, si uno falla, devolverá FAILURE; un nodo de condición, que devuelve SUCCESS o FAILURE dependiendo si la condición se cumple o no. Usando estos 2 nodos junto a una acción ya es posible tener un árbol de comportamiento de lo más simple.

Facilita mucho el crear jerarquía de nodos, ya que un Selector revisa sus hijos de izquierda a derecha buscando uno que devuelva SUCCESS, por ende si el prime hijo tiende a devolver SUCCESS bastante seguido, puede monopolizar los nodos y siempre tomar prioridad, o hacer que opaque ciertos comportamientos, causando errores.

2.3 Estados nuevos: Center y Guard

Tomando esto en cuenta, 2 fantasmas (Pinky e Inky) usarán el modelo BT y, Blinky y Clyde usarán el modelo FSM, además que a cada modelo se le implementó un(a) acción/estado adicional, complejizando un poco más la IA y su toma de decisiones.

2.3.1 Los estados de la IA de los fantasmas

- Scatter: Irse a una esquina del mapa (distinto para cada fantasma).
- Chase: Perseguir a Pacman (Alguno de ellos tienen una forma distinta de seguir a Pacman).
- Frightened: Escapar de Pacman cuando el mismo se come una PowerPill.

2.3.2 Los nuevos estados añadidos

- Para BT, Center: Similar a Scatter, pero en vez de ir a una esquina, se va al centro del mapa. Tiene un 50/50 de suceder en reemplazo de Scatter, haciendolo una variación directa de Scatter.
- Para FSM, Guard: Tiene una similitud directa con Scatter, pero en vez de irse a una esquina, se va a resguardar una PowerPill para evitar que Pacman la agarre. Este no forma parte de un chance de 50/50 como Center, pero forma parte del ciclo principal del fantasma, pasando de "Scatter -> Chase -> 'Repetir'" a "Scatter -> Guard -> Chase -> 'Repetir'". En el caso en el que Pacman se coma la PowerPill que estaba protegiendo pasará automáticamente a Scatter, y si no hay PowerPills disponibles en el mapa, se saltará el estado, volviendo a su ciclo original.

2.3.3 Diagramas finales con los estados nuevos implementados

Diagrama 1: Máquina de Estado Finita (FSM) final

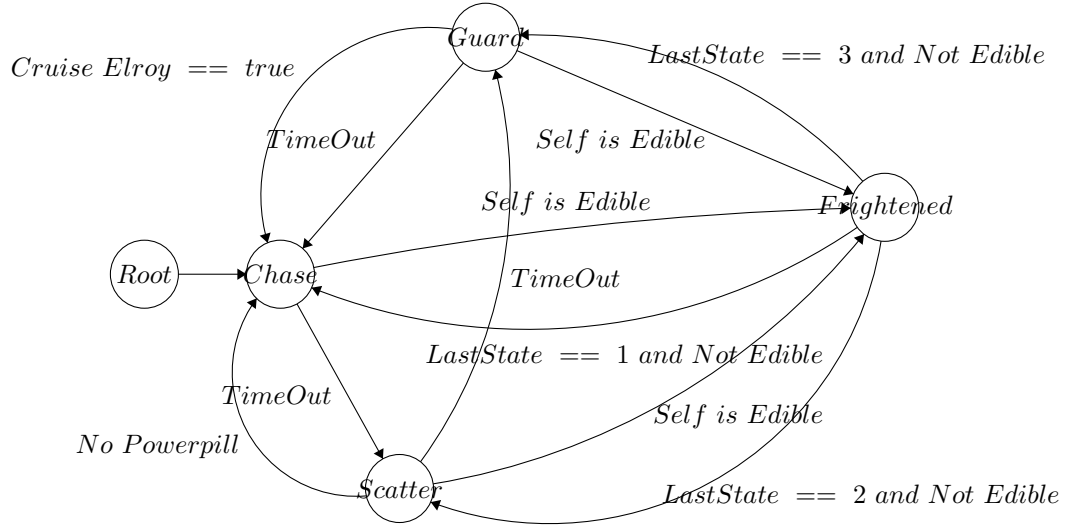
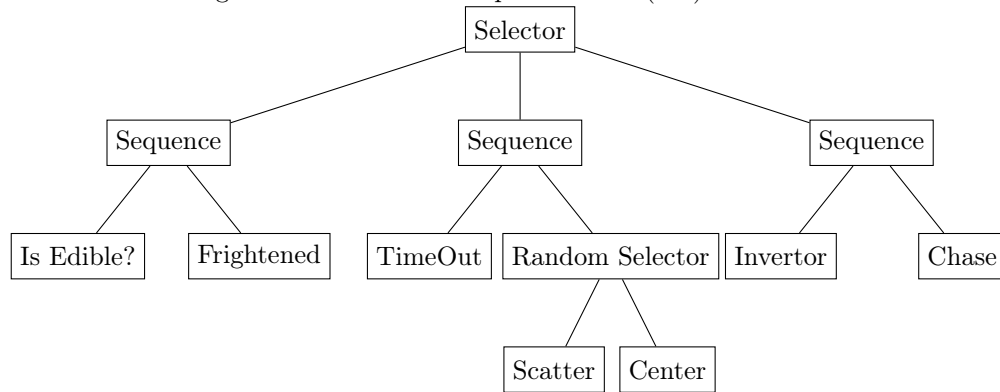


Diagrama 2: Árbol de Comportamiento (BT) final



2.4 Pacman. ¿Máquina de Estado Finita o Árbol de Comportamiento?

Pacman, al ser el personaje que controla el jugador, no tiene una IA como tal, por lo que, para este proyecto, su IA fue implementada usando el modelo de FSM. Esto se debe a la complejidad de sus transiciones, tengo que tener una forma más definida a los estados que puede pasar y a los que no, por supuesto no será perfecto, ni tampoco ganará cada partida que juegue. El modelo no garantiza que la IA sea mejor o no, todo dependen de cómo se implementa y se manejan las jerarquías.

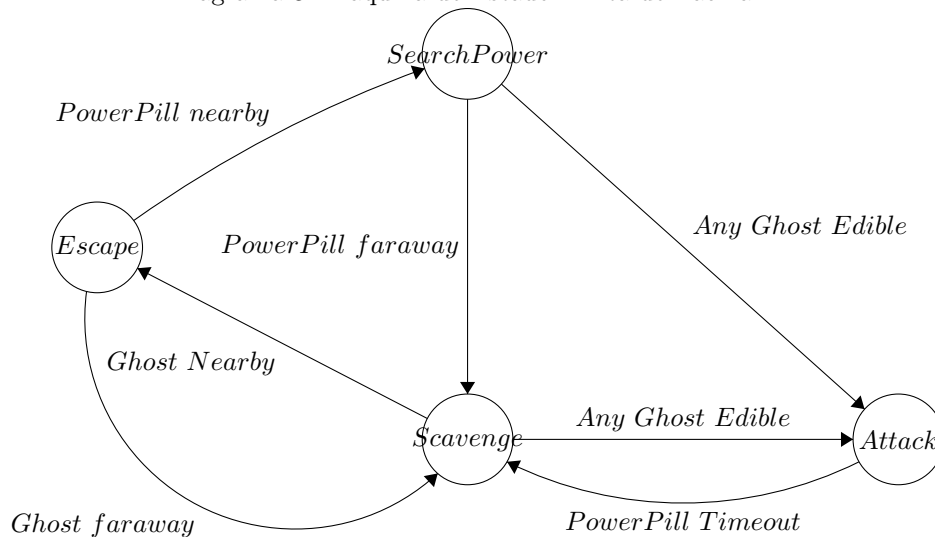
2.4.1 Los estados de Pacman

- SearchPower: Busca la PowerPill más cercana.
- Scavenge: Busca la pildora más cercana. Si hay 2 o 3 pildoras a misma distancia, escogera una aleatoriamente.
- Escape: Escapará del fantasma más cercano.
- Attack: Perseguirá a los fantasmas.

2.4.2 Transiciones de estado de Pacman

- LookForPower: Cumple cuando hay una PowerPill cerca. Es transición de Escape -> SearchPower.
- PowerOutOfRange: Cumple cuando la PowerPill sale del rango de detección. Es transición de SearchPower -> Scavenge.
- LookForPills: Cumple cuando hay una pildora cerca y ninguna fantasma cercano no es comestible. Es transición de Escape -> Scavenge.
- AttackGhosts: Cumple cuando 1 o más fantasmas es(son) comestible(s). Es transición de SearchPower -> Attack y Scavenge -> Attack.
- PowerDown: Cumple cuando ningún fantasma es comestible. Es transición de Attack -> Scavenge.
- RunFromGhosts: Cumple cuando un fantasma está cerca. Es transición de Scavenge -> Escape.

Diagrama 3: Máquina de Estado Finita de Pacman



3 Análisis

Para el análisis de la eficiencia y comportamientos de la IA, se realizaron 25 iteraciones, donde cada iteración era ejecutada con una nueva semilla aleatoria para garantizar que cada iteración sea distinta a la anterior. Si no se maneja esto, todos los valores aleatorios seguirían el mismo patrón siempre que se ejecute el programa.

Los puntos claves que se registraron fueron la cantidad de píldoras que Pacman comió, las PowerPills, cuántos fantasmas alcanzó a comer mientras estaba en el estado de Attack, el puntaje total alcanzado en la iteración, tiempo transcurrido desde comienzo hasta pérdida, y si llegó a activar el "Cruise Elroy" de Blinky. Este último no es realmente un estado, sino una condición que evita que Blinky entre al estado de Scatter y, al nuevo estado añadido, Guard, simulando como si estuviera "enojado".

<i>Valores</i>	Píldoras	PowerPills	Fantasmas	Puntaje	Tiempo (s)
Total	3574	66	35	53040	1314.2
Promedio	143	3	1	2122	52.6
Mejor	218	4	4	3980	NaN
Peor	58	1	0	950	NaN

Hay cosas que aclarar bien en la tabla, los valores de "mejor" y "peor" son el peor valor obtenido entre todas las iteraciones, lo que significa que el peor valor de píldoras no es de la misma iteración del peor valor de fantasmas.

3.0.1 Píldoras

Pacman logró mantener un progreso de píldoras de sobre el 50%, en un caso estuvo a 2 píldoras de completar el mapa, y en otras 3 iteraciones, sólo logró comer bajo 100 píldoras. Se nota la decadencia en efectividad al momento en el que Blinky se enoja, ya que este no para de perseguir a Pacman, que a su vez prioriza escapar de los fantasmas cuando están cerca, haciendo que en 2 iteraciones se formara un ciclo, dónde daban vueltas como un carrusel. Esos 2 casos de carrusel fueron rotos, sólo porque Inky y Pinky mantenían sus estados de Scatter y Center. Además, el mejor valor de las píldoras también muestra que en una sola iteración, Pacman, casi logra completar el nivel, dónde el total de píldoras son 220, fue bastante lamentable esa iteración, pero demuestra que la IA es capaz de completar el nivel, en teoría, pero es muy poco probable.

3.0.2 PowerPills y Fantasmas

Al sólo haber 4 PowerPills, y Pacman siempre empezando en la parte inferior del mapa, siempre estaba cerca de 2 PowerPills facilitando el acceso a ellas y logrando comer mínimo 1 en cada iteración, y en muy pocos casos logró comer las 4. Hay que notar que, no porque haya comido más PowerPills, no significa que haya comido más fantasmas, en cambio, hay iteraciones en las que hay

comido 2-3 PowerPills y no logró comer ningún fantasma. Incluso con el nuevo estado de Guard de Blinky y Clyde, no tendían a defender la PowerPill que estaba más cerca de Pacman, pero en los casos en los que sí, realmente evitaban que Pacman fuera por ellas.

3.0.3 Puntaje

El puntaje se puede calcular muy fácilmente: cada píldora cuenta como 10, las PowerPills cuentan como 50, y los fantasmas cuentan como 400, tristemente no existen las frutas en esta versión de Pacman, al ser una versión bastante simplificada. De todas formas, igual logró tener un promedio de puntaje bastante decente dentro de las limitaciones de la IA. Pero dentro de lo bueno también hay de lo malo, y en este caso esa iteración de Pacman fue tan mala, que sólo logró conseguir 950 puntos consistiendo en 90 píldoras y 1 PowerPill, incluso si en otra iteración sólo comió 58 píldoras y 1 PowerPill, logró comer un fantasma, haciendo que el puntaje sea ligeramente mayor, siendo 1030.

3.0.4 Tiempo

Los tiempos fueron bastante cercanos entre sí, excepto en uno de los ciclos de carrusel que se formó, haciendo que la iteración durara sobre los 2 minutos, de todas formas sólo 8 iteraciones duraron sobre el minuto. Cabe recalcar que a mayor tiempo, no significa mejores resultados, ya que la mejor iteración sólo duró minuto y medio, mientras que las 2 segunda mejores sólo duraron 1 minuto con 4 segundos, y 1 minuto con 14 segundos. Por otro lado, incluso a menor tiempo, tampoco significa peores resultados, debido a que el tiempo más bajo, que fueron 19.4 segundos, Pacman sólo logró comer 73 píldoras, y como se mencionó antes el peor valor de píldoras fue 58, pero esa iteración tuvo una duración de 42.8 segundos. Con todo eso, medir el tiempo sólo sirve para ver cuánto dura la iteración, por ende, el tiempo que dura Pacman vivo.

4 Conclusión final

Mediante el análisis completo de estos comportamientos, claramente hay mucha ventana para mejorar la IA, cómo hacer que Pacman tome en cuenta la posición de todos los fantasmas cercanos al escapara y no sólo de el que está más cerca, o pensar bien sus decisiones si es que un fantasma está en el camino que va a tomar, o si está cerca de la salida del pasillo. Como no se consideraron muchas variables extras, Pacman resultó ser algo torpe y bastante promedio en su trabajo, no resaltando del todo, pero tampoco siendo lo peor de lo peor, haciéndolo suficiente para tareas más simples, que unas más complejas, por ejemplo, si hubieran sólo 1 o 2 fantasmas presentes, Pacman, lo más probable, habría podido de completar el nivel, o incluso los siguientes niveles.

Pacman resultó ser eficiente, pero no eficaz del todo en su tarea, por lo que si se quiere realmente un Pacman que sea imparables, habrá que tomar muchas condiciones extras y complejizar más las transiciones y sus estados, como se

mencionaba en el párrafo anterior. Esto llevaría bastante tiempo para perfeccionarla, muchas pruebas y errores, pero es ahí dónde salen los mejores comportamientos. Mientras más tiempo se le invierte a un agente, comportamientos y transiciones más estables y eficaces tendrá.